

✓ Step 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

```
# =====
# TASK 1 – RESEARCH PAPER IDENTIFICATION
# =====

print("\n===== RESEARCH PAPER =====")

print("Research Paper Title:")
print("Customer Churn Prediction Using Machine Learning Techniques")

print("\nAim:")
print("To predict telecom customer churn using machine learning models.")

print("\nObjectives:")
print("1. Improve customer retention")
print("2. Compare ML algorithms")
print("3. Improve prediction accuracy")

print("\nMethodology:")
print("Data Collection -> Preprocessing -> Model Training -> Evaluation")

print("\nConclusion:")
print("Ensemble methods provide better churn prediction accuracy.")
```

```
===== RESEARCH PAPER =====
Research Paper Title:
Customer Churn Prediction Using Machine Learning Techniques

Aim:
To predict telecom customer churn using machine learning models.

Objectives:
1. Improve customer retention
2. Compare ML algorithms
3. Improve prediction accuracy
```

Methodology:

Data Collection -> Preprocessing -> Model Training -> Evaluation

Conclusion:

Ensemble methods provide better churn prediction accuracy.

▼ Step 2 – Load Dataset

```
data = pd.read_csv("Telco-Customer-Churn.csv")
print(data.head())
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneSer
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CFOCW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection
0	No phone service	DSL	No	...	No
1	No	DSL	Yes	...	Yes
2	No	DSL	Yes	...	No
3	No phone service	DSL	Yes	...	Yes
4	No	Fiber optic	No	...	No

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilli
0	No	No	No	Month-to-month	Y
1	No	No	No	One year	
2	No	No	No	Month-to-month	Y
3	Yes	No	No	One year	
4	No	No	No	Month-to-month	Y

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]

```
print("\n===== DATASET SHAPE =====")
print(data.shape)
```

```
print("\n===== DATASET INFO =====")
print(data.info())
```

```
print("\n===== MISSING VALUES =====")
print(data.isnull().sum())
```

```
===== DATASET SHAPE =====
(7043, 21)
```

```
===== DATASET INFO =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
```

```

#      Column      Non-Null Count  Dtype
---  -
0      customerID    7043 non-null  object
1      gender        7043 non-null  object
2      SeniorCitizen  7043 non-null  int64
3      Partner        7043 non-null  object
4      Dependents     7043 non-null  object
5      tenure         7043 non-null  int64
6      PhoneService   7043 non-null  object
7      MultipleLines   7043 non-null  object
8      InternetService 7043 non-null  object
9      OnlineSecurity  7043 non-null  object
10     OnlineBackup     7043 non-null  object
11     DeviceProtection 7043 non-null  object
12     TechSupport      7043 non-null  object
13     StreamingTV      7043 non-null  object
14     StreamingMovies   7043 non-null  object
15     Contract         7043 non-null  object
16     PaperlessBilling  7043 non-null  object
17     PaymentMethod     7043 non-null  object
18     MonthlyCharges    7043 non-null  float64
19     TotalCharges     7043 non-null  object
20     Churn             7043 non-null  object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

```

```

===== MISSING VALUES =====
customerID    0
gender        0
SeniorCitizen 0
Partner       0
Dependents    0
tenure        0
PhoneService  0
MultipleLines 0
InternetService 0
OnlineSecurity 0
OnlineBackup  0
DeviceProtection 0
TechSupport   0
StreamingTV   0
StreamingMovies 0
Contract      0
PaperlessBilling 0
PaymentMethod 0
MonthlyCharges 0
TotalCharges  0
Churn         0
dtype: int64

```

▼ Step 3 – Remove Unnecessary Column

```
data.drop("customerID", axis=1, inplace=True)
```

▼ Step 4 – Convert Data Type

```
data["TotalCharges"] = pd.to_numeric(
    data["TotalCharges"],
    errors="coerce"
)
```

✓ Step 5 – Handle Missing Values

```
data["TotalCharges"].fillna(
    data["TotalCharges"].median(),
    inplace=True
)
```

/tmp/ipykernel_9147/2560722785.py:1: FutureWarning: A value is trying to
The behavior will change in pandas 3.0. This inplace method will never w

For example, when doing 'df[col].method(value, inplace=True)', try using

```
data["TotalCharges"].fillna(
```

✓ Step 6 – Label Encoding

```
encoder = LabelEncoder()

for column in data.columns:
    if data[column].dtype == "object":
        data[column] = encoder.fit_transform(data[column])
```

✓ Step 7 – Separate Input and Output

```
X = data.drop("Churn", axis=1)
y = data["Churn"]
```

✓ STEP 8 — Feature Scaling

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

✓ Step 9 – Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
```

```
random_state=42
)
```

▼ STEP 10 — Logistic Regression Model

```
lr_model = LogisticRegression(max_iter=1000)
```

```
lr_model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
lr_pred = lr_model.predict(X_test)
```

▼ STEP 11 — Accuracy Calculation

```
lr_accuracy = accuracy_score(y_test, lr_pred)
print(f"Logistic Regression Accuracy: {lr_accuracy*100:.2f}%")
```

```
Logistic Regression Accuracy: 81.55%
```

▼ STEP 12 — Classification Report

```
print(classification_report(y_test, lr_pred))
```

	precision	recall	f1-score	support
0	0.86	0.90	0.88	1036
1	0.68	0.58	0.62	373
accuracy			0.82	1409
macro avg	0.77	0.74	0.75	1409
weighted avg	0.81	0.82	0.81	1409

▼ STEP 13 — Confusion Matrix

```
lr_cm = confusion_matrix(y_test, lr_pred)
```

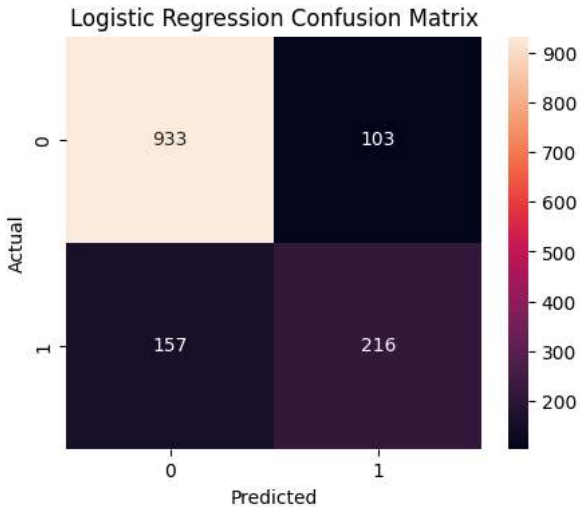
▼ STEP 14 — Heatmap Visualization

```
plt.figure(figsize=(5,4))

sns.heatmap(
    lr_cm,
    annot=True,
    fmt='d'
)

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



▼ STEP 15 — Decision Tree

```
dt_model = DecisionTreeClassifier(max_depth=5)
```

```
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
print(f"Decision Tree Accuracy: {dt_accuracy:.4f}")
```

```
Decision Tree Accuracy: 0.7942
```

```
print("\nDecision Tree Classification Report")
print(classification_report(y_test, dt_pred))
```

```
Decision Tree Classification Report
              precision    recall  f1-score   support

     0               0.87       0.85       0.86       1036
```

	1	0.61	0.64	0.62	373
accuracy				0.79	1409
macro avg		0.74	0.74	0.74	1409
weighted avg		0.80	0.79	0.80	1409

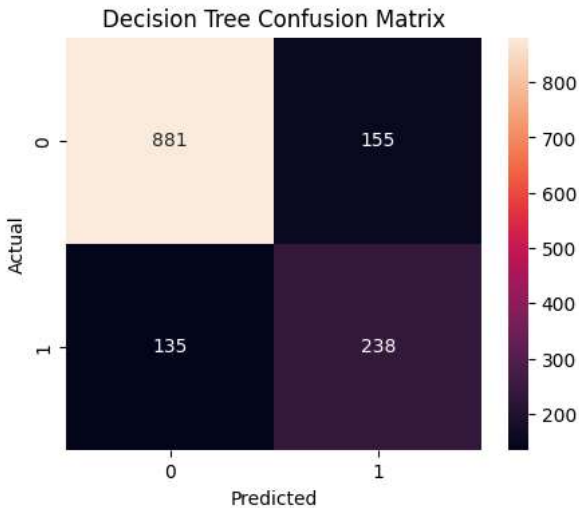
```
dt_cm = confusion_matrix(y_test, dt_pred)

plt.figure(figsize=(5,4))

sns.heatmap(
    dt_cm,
    annot=True,
    fmt='d'
)

plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



▼ STEP 16 — Random Forest

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
```

```
rf_accuracy = accuracy_score(y_test, rf_pred)
print(f"Random Forest Accuracy: {rf_accuracy:.4f}")
```

Random Forest Accuracy: 0.7970

```
print("\nRandom Forest Classification Report")
print(classification_report(y_test, rf_pred))
```

```
Random Forest Classification Report
```

	precision	recall	f1-score	support
0	0.83	0.91	0.87	1036
1	0.66	0.48	0.55	373
accuracy			0.80	1409
macro avg	0.75	0.69	0.71	1409
weighted avg	0.78	0.80	0.79	1409

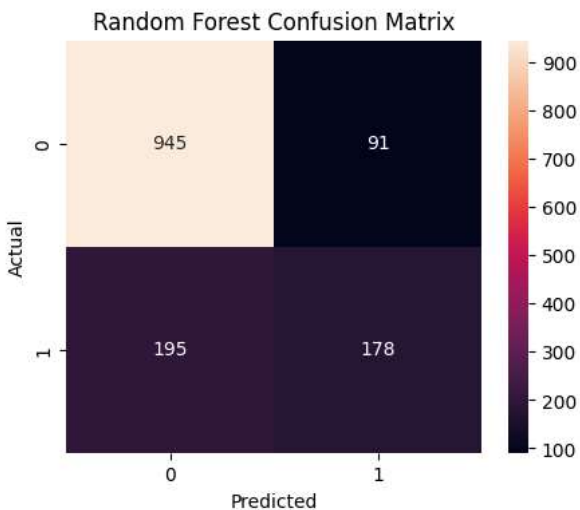
```
rf_cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(5,4))

sns.heatmap(
    rf_cm,
    annot=True,
    fmt='d'
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



▼ STEP 17 — XGBoost

```
xgb_model = XGBClassifier(  
    eval_metric='logloss',  
    use_label_encoder=False  
)
```

```
xgb_model.fit(X_train, y_train)  
xgb_pred = xgb_model.predict(X_test)  
xgb_accuracy = accuracy_score(y_test, xgb_pred)  
print(f"XGBoost Accuracy: {xgb_accuracy:.4f}")
```

XGBoost Accuracy: 0.7942
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: Parameters: { "use_label_encoder" } are not used.

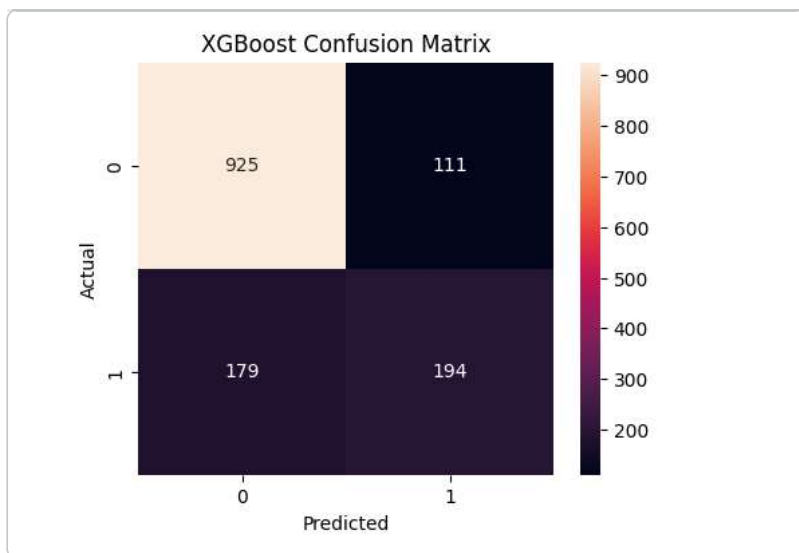
```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
print("\nXGBoost Classification Report")  
print(classification_report(y_test, xgb_pred))
```

```
XGBoost Classification Report
```

	precision	recall	f1-score	support
0	0.84	0.89	0.86	1036
1	0.64	0.52	0.57	373
accuracy			0.79	1409
macro avg	0.74	0.71	0.72	1409
weighted avg	0.78	0.79	0.79	1409

```
xgb_cm = confusion_matrix(y_test, xgb_pred)  
  
plt.figure(figsize=(5,4))  
  
sns.heatmap(  
    xgb_cm,  
    annot=True,  
    fmt='d'  
)  
  
plt.title("XGBoost Confusion Matrix")  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
  
plt.show()
```



▼ STEP 18 — Model Comparison Table

```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest",
              "XGBoost"],
    "Accuracy": [lr_accuracy, dt_accuracy, rf_accuracy, xgb_accuracy]
})
display(comparison)
```

	Model	Accuracy	
0	Logistic Regression	0.815472	
1	Decision Tree	0.794180	
2	Random Forest	0.797019	
3	XGBoost	0.794180	

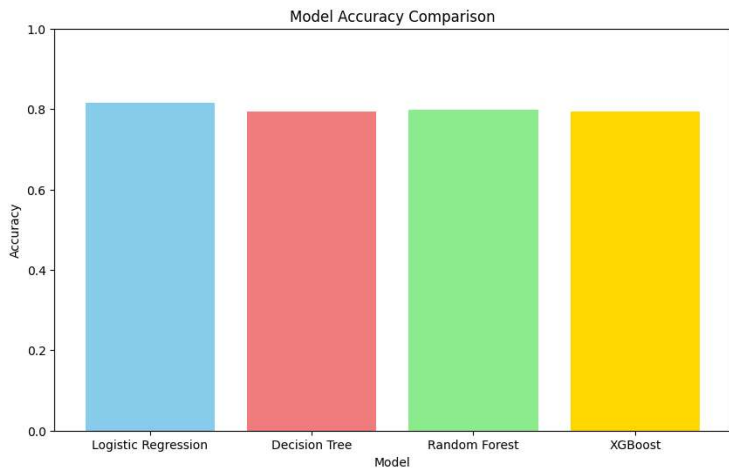
Next steps: [Generate code with comparison](#)

[New interactive sheet](#)

▼ STEP 19 — Accuracy Graph

```
plt.figure(figsize=(10, 6))
plt.bar(
    comparison["Model"],
    comparison["Accuracy"],
    color=['skyblue', 'lightcoral', 'lightgreen', 'gold']
)
plt.xlabel("Model")
plt.ylabel("Accuracy")
```

```
plt.title("Model Accuracy Comparison")
plt.ylim(0.0, 1.0) # Set y-axis limit for accuracy
plt.show()
```



✓ STEP 20 — Feature Importance

```
importance = pd.DataFrame({
    "Feature": data.drop("Churn", axis=1).columns,
    "Importance": rf_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\n===== FEATURE IMPORTANCE =====")
print(importance)

plt.figure(figsize=(10,6))

sns.barplot(
    x="Importance",
```

```

        y="Feature",
        data=importance
    )

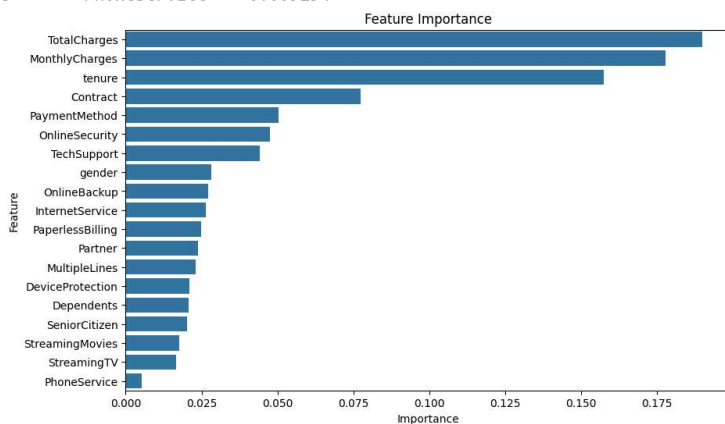
plt.title("Feature Importance")

plt.show()

```

===== FEATURE IMPORTANCE =====

	Feature	Importance
18	TotalCharges	0.189992
17	MonthlyCharges	0.177867
4	tenure	0.157428
14	Contract	0.077357
16	PaymentMethod	0.050379
8	OnlineSecurity	0.047432
11	TechSupport	0.044190
0	gender	0.028117
9	OnlineBackup	0.027067
7	InternetService	0.026444
15	PaperlessBilling	0.024917
2	Partner	0.023864
6	MultipleLines	0.023131
10	DeviceProtection	0.021012
3	Dependents	0.020709
1	SeniorCitizen	0.020354
13	StreamingMovies	0.017737
12	StreamingTV	0.016749
5	PhoneService	0.005254



✓ Step 21 -Customer Churn Status

```
sample_prediction = xgb_model.predict(X_test[:10])

print("\nActual Values:")
print(y_test[:10].values)

print("\n===== CUSTOMER CHURN STATUS =====")

for i in range(10):

    if sample_prediction[i] == 1:
        print(f"Customer {i+1} : Churn")

    else:
        print(f"Customer {i+1} : No Churn")
```

Actual Values:
[1 0 0 1 0 1 0 0 1 1]

===== CUSTOMER CHURN STATUS =====
Customer 1 : Churn
Customer 2 : No Churn
Customer 3 : No Churn
Customer 4 : Churn
Customer 5 : No Churn
Customer 6 : No Churn
Customer 7 : No Churn
Customer 8 : No Churn
Customer 9 : No Churn
Customer 10 : No Churn

```
sample_prediction = xgb_model.predict(X_test[:10])

print(f"Predicted Values:\n{sample_prediction}")

print("\nActual Values:")
print(y_test[:10].values)

print("\n===== CUSTOMER CHURN STATUS =====")

for i in range(10):

    if sample_prediction[i] == 1:
        print(f"Customer {i+1} : Churn")

    else:
        print(f"Customer {i+1} : No Churn")
```

Predicted Values:
[1 0 0 1 0 0 0 0 0 0]

Actual Values:
[1 0 0 1 0 1 0 0 1 1]

===== CUSTOMER CHURN STATUS =====

```
Customer 1 : Churn
Customer 2 : No Churn
Customer 3 : No Churn
Customer 4 : Churn
Customer 5 : No Churn
Customer 6 : No Churn
Customer 7 : No Churn
Customer 8 : No Churn
Customer 9 : No Churn
Customer 10 : No Churn
```

▼ Step 22 - Overfitting/Underfitting Analysis

```
print("\n===== MODEL ANALYSIS =====")

print("Logistic Regression : Slight underfitting")
print("Decision Tree       : May overfit")
print("Random Forest        : Better generalization")
print("XGBoost               : Best bias-variance balance")
```

```
===== MODEL ANALYSIS =====
Logistic Regression : Slight underfitting
Decision Tree       : May overfit
Random Forest        : Better generalization
XGBoost             : Best bias-variance balance
```

▼ STEP 23 — Final Result

```
X_new = X_test[0:1] # Using the first sample from X_test as an example

pred = lr_model.predict(X_new)
prob = lr_model.predict_proba(X_new)

print(f"Churn: {pred[0]}")
print(f"Probability: {prob[0][1]:.2%}")
```

```
Churn: 1
Probability: 65.15%
```

```
best_model = comparison.loc[
    comparison["Accuracy"].idxmax(),
    "Model"
]
best_accuracy = comparison["Accuracy"].max()
print(f"Best Model : {best_model}")
print(f"Best Accuracy : {best_accuracy:.2f}")
```

```
Best Model : Logistic Regression
Best Accuracy : 0.82
```

```
# =====
# LIVE CUSTOMER CHURN PREDICTION SYSTEM
# =====
```

```

while True:

    print("\n===== CUSTOMER CHURN CHECK =====")

    # USER INPUT

    tenure = int(input("Enter Tenure (months): "))

    monthly_charges = float(input("Enter Monthly Charges: "))

    contract = int(input(
        "Contract Type (0=Month-to-month, 1=One year, 2=Two year): "
    ))

    # =====
    # CREATE CUSTOMER DATA
    # =====

    new_customer = [[

        1,                # gender
        0,                # SeniorCitizen
        1,                # Partner
        0,                # Dependents
        tenure,
        1,                # PhoneService
        0,                # MultipleLines
        2,                # InternetService
        0,                # OnlineSecurity
        1,                # OnlineBackup
        0,                # DeviceProtection
        0,                # TechSupport
        1,                # StreamingTV
        1,                # StreamingMovies
        contract,
        1,                # PaperlessBilling
        2,                # PaymentMethod
        monthly_charges,
        monthly_charges * tenure

    ]]

    # =====
    # APPLY SCALING
    # =====

    new_customer_scaled = scaler.transform(new_customer)

    # =====
    # PREDICT CHURN
    # =====

    prediction = xgb_model.predict(new_customer_scaled)

    # =====
    # SHOW RESULT
    # =====

```

```

print("\n===== RESULT =====")

if prediction[0] == 1:

    print("Customer Will Churn")

else:

    print("Customer Will NOT Churn")

# =====
# ASK USER TO CONTINUE
# =====

choice = input("\nDo You Want To Check Another Customer? (yes/no): "

if choice.lower() == "no":

    print("\n===== SYSTEM CLOSED =====")
    print("Thank You")

    break

```

```

===== CUSTOMER CHURN CHECK =====
Enter Tenure (months): 2
Enter Monthly Charges: 99
Contract Type (0=Month-to-month, 1=One year, 2=Two year): 2
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739
  warnings.warn(

===== RESULT =====
Customer Will NOT Churn

Do You Want To Check Another Customer? (yes/no): no

===== SYSTEM CLOSED =====
Thank You

```